Inverting Python: Designing NTP-Prior-Resistant Programming Languages for LLM-Resistant Coding Assessment

Jaeyeong CHOI
DGIST EECS
jaeyeong2022@dgist.ac.kr

***Abstract*—LLMs now solve standard programming tasks well enough to undermine their value as student assessments. We investigate whether it is possible to design a programming language that is human-interpretable but defeats LLM next-token-prediction (NTP) priors. We propose a four-level taxonomy—L1 (token alias substitution), L2 (syntax inversion), L3 (semantic inversion with explicit rules), L4 (semantic inversion with implicit delivery)—and evaluate LLM compliance across seven model families (`gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`). L1 creates partial resistance (KLR 0.26–0.69); L2 is easily adopted; L3 reveals a model capability threshold; and L4 defeats all tested models on the Fibonacci task through *pattern blindness*, with task-dependent outcomes on simpler idioms. A multi-task extension (Fibonacci, is_sorted, count_vowels; 395 runs) reveals task-dependent resistance and a third failure mode, *operational substitution*. We identify three failure mechanisms—*prior dominance*, *pattern blindness*, and *operational substitution*—across 825+ total L4 evaluations spanning 9 models (7 OpenAI, 2 open-weight). A chain-of-thought ablation reveals that models can *articulate* the inversion rule when prompted to reason but still generate standard Python code, demonstrating a reasoning–generation dissociation. Preliminary open-weight evaluation (`Llama-3.3-70B`, `Qwen3-32B`) shows consistent L4 pattern blindness and wide L1 compliance variance (KLR=0.23 vs. 1.00), suggesting architecture and fine-tuning regime govern alias resistance.**

***Index Terms*—confusion language, LLM resistance, NTP prior, semantic inversion, programming education, code assessment**

I. Introduction

Modern code-generation benchmarks [1], [2] show that large language models (LLMs) can solve standard programming tasks with high accuracy. This creates a critical challenge for programming education: if LLMs solve assignments faster and more accurately than students, how can instructors meaningfully assess student understanding?

This paper studies a complementary but understudied question:

> *Is it possible to design a programming language that is human-interpretable but systematically defeats LLM next-token-prediction priors—making it resistant to AI-based code generation?*

We decompose this into three sub-questions:

1) Does changing token identifiers (L1: alias substitution) create effective LLM resistance?
2) Does inverting syntactic structure (L2) create effective resistance?
3) Does inverting semantic behavior (L3/L4) create deeper resistance?

Our key empirical findings are:

- **L1 (token substitution):** Partially effective—keyword leakage rate (KLR) ranges from 0.26 to 0.69 across models. Models revert to Python priors at measurable rates.
- **L2 (syntax inversion):** Not effective—models adopt new syntax completely (SIR=0.0) when given explicit rules.
- **L3 (semantic inversion, explicit):** Capacity-dependent—weaker models fail completely (PPR=1.0 for `gpt-4o-mini`), while stronger models succeed (PPR=0.0 for `gpt-4o` and `gpt-4.1-mini`).
- **L4 (semantic inversion, implicit):** Effective against all tested models—even strong models exhibit *pattern blindness*, ignoring semantic inversion cues in examples.

We also identify three distinct LLM failure modes: *prior dominance*, where the prior overrides stated rules; *pattern blindness*, where the model fails to extract semantic rules from examples; and *operational substitution*, where the model achieves correct output via arithmetic workaround rather than adopting the semantic rule.[1]

This framework provides both a theoretical taxonomy for LLM-resistant language design and practical tools for constructing programming assessments that remain meaningful in the LLM era.

II. Related Work

A. Code Generation Benchmarks and Evaluation

HumanEval [1] and MBPP [2] established functional correctness as the standard metric for code-generation evaluation. More recent benchmarks scale and diversify tasks: BigCodeBench [3] adds complex, multi-function instructions, and LiveCodeBench [4] avoids contamination through continuous updating. Across all these benchmarks, Python dominates training corpora, embedding strong token-level priors into all models evaluated. This motivates our L1 baseline: even with explicit alias instructions, models leak Python keywords at 26–69%.

[1]Operational substitution was observed on `count_vowels` for `gpt-4o` at $n = 50$.

B. Code-Oriented LLM Architectures and Training

Pre-trained code models—CodeBERT [5], CodeT5 [6], and InCoder [7]—demonstrate that NTP objectives on Python code reinforce keyword position priors. More recent instruction-tuned models (WizardCoder [8], DeepSeek-Coder [9]) amplify these priors through even larger code corpora. This trajectory predicts our finding: *stronger code LLMs do not necessarily mean better alias compliance*—gpt-5.4-mini (KLR=0.67) underperforms gpt-4o (KLR=0.37).

C. Instruction-Following in Code (2024–2025)

CodeIF [10] and MultiCodeIF [11] introduce benchmarks measuring LLM adherence to developer-specified constraints in code generation tasks. Critically, MultiCodeIF reports that models perform relatively well on *explicit, concrete* constraints but degrade substantially on *implicit or abstract* constraints, and that performance drops sharply as the number of hierarchical constraints increases. This explicit–implicit asymmetry is the closest mechanistic evidence in the literature to our L3/L4 findings: models can follow explicit semantic inversion rules (L3, with some model-capacity dependence), yet fail entirely when the same semantic inversion is delivered only through examples without explicit articulation (L4, pattern blindness across all tested models). The key distinction is scope: CodeIF and MultiCodeIF benchmark *constraint-following* within standard programming tasks, whereas our work *designs* a programming language whose semantic structure systematically conflicts with NTP priors, making resistance a structural property rather than a task-level benchmark artifact.

D. Instruction Hierarchy and Prior Conflicts

Chen et al. [12] propose an instruction hierarchy to resolve conflicts between system-level and user-level instructions. Their framework addresses natural-language directive conflicts; ours involves programming-language semantic rule conflicts—a dimension not studied in alignment work. Xie et al. [13] and Wu et al. [14] show that when context conflicts with parametric knowledge, LLMs often default to pretraining priors. Zhang et al. [15] survey this knowledge conflict problem broadly. Shi et al. [16] demonstrate LLMs are easily distracted by irrelevant context; our L4 condition represents the inverse: models *fail to use* relevant semantic cues provided in examples.

E. NTP Priors and In-Context Learning

Wang et al. [17] formalize a "law of next-token prediction" providing theoretical grounding for how NTP training shapes output distributions. Min et al. [18] show ICL demonstrations affect outputs primarily through distributional surface patterns rather than semantic content—explaining why models fail to extract inverted semantics from L4 examples. Dong et al. [19] survey ICL comprehensively. Lampinen et al. [20] document "A-Not-B" errors where LLMs fail to suppress previously established response patterns under context changes, directly analogous to our L4 pattern blindness.

F. Counterfactual and Semantic Reasoning Failures

Zhao et al. [21] show LLMs struggle when counterfactual reasoning requires overriding parametric knowledge—the same mechanism we observe at L3 for weak models. Turpin et al. [22] show LLM self-explanations are often unfaithful to actual behavior. Valmeekam et al. [23] document reasoning rigidity—models revert to memorized strategies under redefined rules—providing further precedent for our pattern blindness finding.

G. Adversarial Perturbation and Code Resistance

PromptBench [24] demonstrates LLM fragility under text perturbations. Pan et al. [25] show semantically aligned code prompt perturbations cause up to 42% accuracy drops. Jha et al. [26] use structured adversarial prompts to break LLM reasoning in code tasks. Khan et al. [27] specifically design prompt perturbations to reduce LLM correctness on programming assignments by 77%, most directly related to our goal. Our approach is complementary: instead of perturbing prompts, we design a confusion language whose NTP prior conflict is a *structural property*, making it principled and reproducible.

H. DSL and Constrained Code Generation

Wu et al. [28] survey LLM performance on low-resource and domain-specific languages, confirming that Python-trained models underperform on less common languages. ConCodeEval [29] benchmarks code constraint comprehension in DSLs; L2CEval [30] evaluates language-to-code generation across specialized languages. Constrained decoding [31], [32] enforces grammar-level output validity. Our L2 experiment shows models adopt explicit syntax rules entirely, but semantic constraints (L3/L4) prove far more resistant—showing grammar-level constraints are insufficient without semantic-level challenges.

I. Real-World Engineering Benchmarks

SWE-bench [33] reveals that LLMs fail to generalize from canonical benchmarks to real repository-level tasks—confirming that high performance on standard benchmarks does not indicate robustness to structural novelty, as we observe in our L1 L4 evaluations.

J. LLM in Programming Education

Dakhel et al. [34] show LLM coding tools are an "asset or liability" depending on student expertise. Lau and Guo [35] document novice programmers adopting LLM outputs uncritically. Prather et al. [36] show mixed student perceptions. Liffiton et al. [37] propose guardrailed LLM tutoring. These studies confirm a shared concern: LLM integration can undermine independent skill development.

K. Academic Integrity and LLM-Resistant Assessment

Novak and McDanel [38] propose contextual, personalized assignments to resist LLM cheating—the most directly related paper to our educational goal—but without a formal theory of what features drive LLM failure. Code perplexity detection [39]

TABLE I: Four-level confusion language taxonomy. Each level targets a distinct linguistic dimension and delivery modality. L4 is the only level where all tested models fail regardless of capacity.

Level	Perturbation	Delivery	Observed failure mode
L1	Token alias	Explicit	Prior dominance (partial)
L2	Syntax inv.	Explicit	None—adopted
L3	Semantic inv.	Explicit	Capacity-dependent
L4	Semantic inv.	Implicit	Pattern blindness

is brittle; our taxonomy offers a principled design-based solution. RLHF-based alignment [40] improves instruction-following but, as our L3/L4 results show, remains insufficient when semantic inversions must be extracted from examples.

L. Research Gap

Prior adversarial work degrades generation quality without principled language design. Prior DSL work measures performance without explaining failure mechanisms. Prior knowledge-conflict work focuses on factual knowledge, not programming-language semantics. Our contribution closes these gaps: a theory-grounded L1–L4 taxonomy that characterizes LLM failure by mechanism (prior dominance vs. pattern blindness) and guides principled LLM-resistant language design.

III. Method

A. Research Questions and Hypotheses

RQ1: At what language design level does LLM resistance become robust? **RQ2**: What failure mechanism is active at each level—prior dominance or pattern blindness?

H1: Token-level alias substitution (L1) creates partial but not complete resistance. **H2**: Syntactic inversion (L2) is insufficient—models adopt new syntax when given explicit rules. **H3**: Semantic inversion (L3/L4) provides robust resistance, but differs in mechanism by model capacity.

B. Language Level Taxonomy

We design a four-level taxonomy along two axes: (1) *perturbation depth* (token, syntax, semantic) and (2) *rule delivery* (explicit vs. implicit). Table I summarizes each level.

Figure 1 illustrates the two-axis design space and the location of each level. The two gray cells (Token/Implicit and Syntax/Implicit) are theoretically possible configurations that we did not study; Token/Implicit would expect higher KLR than L1-Explicit (weaker contextual cues), while Syntax/Implicit would expect higher SIR. We leave these as open positions in the design space for future work.

C. L1: Token Alias Substitution

Python keywords (`if`, `for`, `in`, `return`, `def`) are replaced with aliases. Constraints: one-to-one mapping, no alias collision, token-boundary replacement, literal/comment protection.

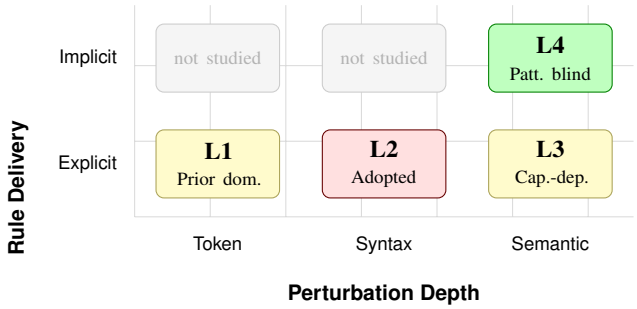

Fig. 1: Two-axis confusion language design space. Perturbation depth (token/syntax/semantic) and rule delivery (explicit/implicit) together determine LLM failure mode. L4 (dark red) occupies the hardest cell: semantic perturbation without explicit rule—the only condition where all tested models fail.

Python (standard)

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
```

Standard semantics: if-block runs when condition is TRUE

L3 (semantic inversion)

```
def fib(n):
    if n <= 1:
        return fib(n-1) + fib(n-2)
    return n
```

Inverted: if-block runs when condition is FALSE

Fig. 2: Semantic inversion in L3. Surface syntax is identical to Python, but if-block semantics are inverted. The correct L3 solution (right) looks counter-intuitive to Python-trained models, which generate the left version regardless of the stated rule.

D. L2: Syntax Structure Inversion

Function definitions, conditionals, loops, and block delimiters are structurally inverted relative to Python. Example: `def fib(n):` becomes `:define fib [n] ->`. Model is given explicit grammar rules.

E. L3: Semantic Inversion (Explicit Rules)

Python surface syntax is retained but the semantics of if-blocks are inverted: the indented block executes when the condition is *false*. Model is explicitly told this rule. Figure 2 illustrates the concept.

F. L4: Semantic Inversion (Implicit, Example-Only)

Same as L3, but no rule is stated. Instead, models receive 2–3 worked examples that demonstrate the inversion. Models must extract the semantic pattern through few-shot reasoning.

G. Evaluation Metrics

Redundant metrics are removed. We report two independent measures:

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. Note: ACR $= 1 -$ KLR and PIR $\approx$ KLR are derived quantities and not separately reported.
- **PSS** (Partial Structural Score, exploratory): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1). Weights are heuristic; PSS is reported as a secondary diagnostic alongside KLR and should not be used as the sole basis for compliance conclusions.
- **PPR** (Python Prior Rate, L3/L4): fraction of samples using Python-style condition (no inversion).

H. Threats to Validity

Internal: judge rules may be stricter than human acceptance; PSS weights are heuristic. **Construct**: multi-task extension (T1 fib, T2 is_sorted, T3 count_vowels) partially addresses single-task limitation; broader coverage (nested control-flow, stateful logic) and open-weight model replication remain for future work. **External**: results cover seven OpenAI-family models (`gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`); open-weight and non-OpenAI model replication is future work.

IV. Experiments

A. Experimental Overview

We ran experiments at each language level for available model families. All runs used prompts from `v1-v120` (L1) or dedicated level-specific banks (L2–L4). API authentication was managed via project environment variables.

B. Models Evaluated

- `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`

C. L1 Experiment: Token Alias Substitution

120 prompts per model, versioned v1–v120 with varied alias mappings. Context-pack ablation: baseline vs. explicit alias contract system prompt.

D. L2 Experiment: Syntax Inversion

5 prompts $\times$ 4 runs each ($n = 20$) using `:define ... ->` syntax (see Appendix). Model given complete grammar specification. Evaluated for `gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano` (plus $n = 5$ for `gpt-4o-mini` from earlier run).

E. L3 Experiment: Semantic Inversion (Explicit)

10 prompts $\times$ 2 runs each ($n = 20$). System message explicitly states: "if-blocks execute when condition is FALSE." Evaluated for `gpt-4o`, `gpt-4.1-mini`, `gpt-4.1-nano` (plus $n = 5$–10 for `gpt-4o-mini` and `gpt-4o` from earlier runs).

F. L4 Experiment: Semantic Inversion (Implicit)

Initial screening: 10 prompts per model, no rule stated; only 2 worked examples with inverted semantics shown. Model must infer pattern. Ablation (n=50): 5 prompt variants $\times$ 10 runs each, varying the number of worked examples (2 vs. 3), presence of inline comments, order of examples, and domain of examples. Evaluated across all seven models at $n = 50$ each (350 total L4 ablation runs): `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, and `o4-mini`.

G. L4 Multi-task Generalization Experiment

To assess whether pattern blindness generalizes across algorithmic domains, we extended the L4 evaluation to two additional tasks beyond Fibonacci: **T2 (is_sorted)**: write a function returning True if a list is ascending, and **T3 (count_vowels)**: count vowel characters in a string. Both use the identical shared prompt header (two worked examples with inverted semantics, no explicit rule)—the same prompt structure as Ablation Variant A. We ran $n = 10$ per task for `gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-nano`, and `gpt-5.4-mini`; $n = 20$ for `gpt-4.1-mini`; and $n = 15$ for `o4-mini` (255 initial runs). We then scaled `gpt-4o` T3 and `gpt-4.1` T2 to $n = 50$ to resolve preliminary findings, yielding 355 total multi-task runs. Results are in Table V.

H. CoT Ablation Experiment

To test whether chain-of-thought reasoning overcomes pattern blindness, we added a CoT instruction ("think step by step about the pattern first, then write code") to the L4 Variant A prompt. We compared CoT vs. no-CoT for `gpt-4o` and `gpt-4.1-mini` ($n = 20$ per condition, 80 total runs). Judge criteria are identical to the L4 ablation; we additionally track whether the model's reasoning text mentions the inversion pattern.

I. Reproducibility

All experimental artifacts, commands, and results are committed to the repository under `docs/research/results/`. See reproducibility protocol section for commands.

V. Results

A. L1: Partial resistance; scale does not predict compliance

All OpenAI models show non-zero KLR (0.21–0.69), confirming prior dominance. Three findings stand out.

(1) `gpt-5.4-mini` (KLR=0.67) underperforms `gpt-4o` (KLR=0.37) despite being a newer model; because these models differ in training corpus and RLHF regime—not just scale—we characterize this as *model-family-specific compliance patterns* rather than a scale effect. The observed difference may reflect stronger code-generation fine-tuning in `gpt-5.4-mini` (a stated hypothesis, not a measured mechanism).

(2) For models with paired conditions, context-pack delivery reduces KLR: `gpt-4o-mini` ($0.42 \rightarrow 0.26$, -38%) and `gpt-4.1-mini` ($0.43 \rightarrow 0.21$, -51%).

TABLE II: L1 evaluation: KLR and PSS per model. Baseline rows use $n = 120$; ctx-pack rows for `gpt-4.1` family, `o4-mini`, and open-weight models use $n = 20$. Models sorted by KLR (ascending). For `gpt-4o-mini` and `gpt-4.1-mini` (paired conditions), context-pack reduces KLR (-38% and -51% respectively). Open-weight models (marked *) are ctx-pack only via Groq API.

Model	Delivery	KLR↓	PSS↑
gpt-4.1-mini	ctx-pack	**0.21**	0.37
Llama-3.3-70B*	ctx-pack	0.23	0.34
gpt-4o-mini	ctx-pack	0.26	0.32
o4-mini	ctx-pack	0.27	0.41
gpt-4.1	ctx-pack	0.30	**0.43**
gpt-4o	baseline	0.37	**0.40**
gpt-4.1-nano	ctx-pack	0.38	0.30
gpt-4o-mini	baseline	0.42	0.27
gpt-4.1-mini	baseline	0.43	0.33
gpt-5.4-mini	baseline	0.67	0.25
gpt-4.1-nano	baseline	0.69	0.16
Qwen3-32B*	ctx-pack	1.00	0.00

*Open-weight via Groq API; $n = 20$, ctx-pack only. See §V-H.

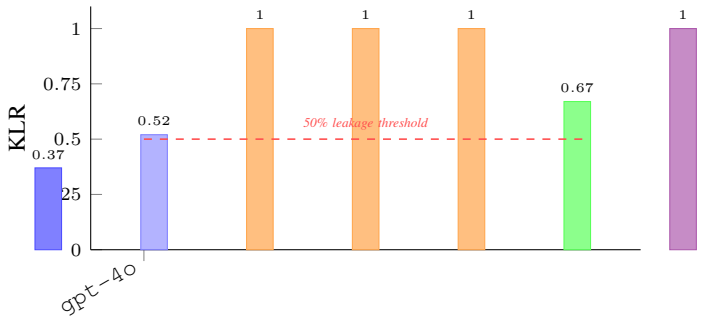

Fig. 3: L1 Keyword Leakage Rate (KLR) by model. Lower is better (less Python keyword leakage). Color indicates model family: blue = `gpt-4o`, orange = `gpt-4.1`, green = `gpt-5`, purple = `o4`. Dashed line marks 50% leakage threshold.

(3) The reasoning model `o4-mini` (KLR=0.27) performs comparably to `gpt-4o-mini` ctx-pack (0.26), suggesting chain-of-thought reasoning provides modest benefit for alias compliance.

Note: `gpt-4.1`, `gpt-4.1-nano`, and `o4-mini` were only evaluated under ctx-pack ($n = 20$, v1–v20); baseline comparison for these models is not available and should be interpreted with caution given the smaller and non-overlapping prompt subset.

Open-weight preliminary: `Llama-3.3-70B` (KLR=0.23) is comparable to the best OpenAI ctx-pack results; `Qwen3-32B` (KLR=1.00) is the worst result across all tested models. This cross-family variance (0.23 vs. 1.00) exceeds the spread within the `gpt-4.x` family, indicating that architecture and instruction-following fine-tuning—not parameter count—govern alias compliance. See §V-H for full details and limitations.

B. L2: Syntax inversion not resistant

All four evaluated models (`gpt-4o`, `gpt-4o-mini`, `gpt-4.1-mini`, `gpt-4.1-nano`) adopted L2 syntax fully (SIR=0.0, $n = 20$ each), confirming that structural novelty alone does not create LLM resistance.

TABLE III: L2–L4 outcomes by model and condition. L2 is trivially adopted by all models ($n = 20$); L3 reveals a model-capability threshold (`gpt-4o-mini` fails, others succeed); L4 (example-only) defeats all models despite L3 success.

Model	Level	Delivery	Pass	Failure mode
gpt-4o-mini	L2	Expl. grammar	5/5	None (adopted)
gpt-4o	L2	Expl. grammar	20/20	None (adopted)
gpt-4.1-mini	L2	Expl. grammar	20/20	None (adopted)
gpt-4.1-nano	L2	Expl. grammar	20/20	None (adopted)
gpt-4o-mini	L3	Expl. rule	0/5	Prior dominance
gpt-4o	L3	Expl. rule	20/20	— (success)
gpt-4.1-mini	L3	Expl. rule	20/20	— (success)
gpt-4.1-nano	L3	Expl. rule	20/20	— (success)
gpt-4o	L4	Examples only	0/5	Pattern blindness
gpt-4.1-nano	L4	Examples only	0/50	Pattern blindness

C. L3: Capability threshold at semantic rule-following

With an explicit semantic inversion rule, `gpt-4o-mini` fails completely (PPR=1.0): it generates standard Python despite instruction. By contrast, `gpt-4o`, `gpt-4.1-mini`, and `gpt-4.1-nano` all succeed (20/20, PPR=0.0). This reveals a model capacity threshold between `gpt-4o-mini` and the larger/newer models for following semantic meta-rules.

Key contrast: L3 success $\not\Rightarrow$ L4 success. `gpt-4.1-nano` achieves 20/20 at L3 but 0/50 (PPR=1.0) at L4—the same model that correctly follows an explicit inversion rule is completely blind to the same rule embedded only in examples. This within-model contrast directly isolates pattern blindness as a failure mode independent of model capacity to follow rules when stated explicitly.

D. L4: Pattern blindness across all tested models

Without explicit rule, `gpt-4o`—which succeeds at L3—fails at L4 (PPR=1.0), producing canonical Python and ignoring example-embedded inversion cues. Figure 4 shows the contrast.

L4 ablation (n=50 per model). We scaled the L4 evaluation across seven models, all at $n = 50$ (`gpt-4o`, `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`, `gpt-5.4-mini`, `o4-mini`). Results are shown in Table IV. All seven models achieve 0% pass rate (0/50 each), confirming that pattern blindness is robust across model families and including the reasoning model `o4-mini`. PPR values range from 0.38 (`gpt-4.1-mini`) to 1.0 (`gpt-4.1`, `gpt-4.1-nano`, `gpt-5.4-mini`); `o4-mini` shows PPR=0.92. Notably, `gpt-4.1` and `gpt-4.1-nano` show PPR=1.0 across *all* 50 runs and all variants, indicating complete prior dominance with no evidence of inversion detection. The `o4-mini` result revises an earlier preliminary finding (1/20 at $n = 20$): at the full scale of $n = 50$, `o4-mini` also achieves 0/50, indicating that chain-of-thought reasoning does not provide a reliable pathway to overcoming pattern blindness at this scale.

TABLE IV: L4 ablation (PPR = Python Prior Rate). All seven models: $n = 50$ each (5 variants $\times$ 10 runs). Pass rate = 0/50 for all models. `o4-mini` (reasoning model) also achieves 0/50 at $n = 50$, revising a preliminary $n = 20$ result (1/20).

Var.	Description	gpt-4o	gpt-4o-mini	gpt-4.1	gpt-4.1-mini	gpt-4.1-nano	gpt-5.4-mini	o4-mini
A	2ex-baseline	0.40	**0.10**	1.00	**0.10**	1.00	1.00	0.80
B	3ex-extra	1.00	1.00	1.00	**0.00**	1.00	1.00	1.00
C	2ex-no-comment	0.90	1.00	1.00	0.80	1.00	1.00	0.90
D	2ex-swapped	0.70	1.00	1.00	**0.00**	1.00	1.00	1.00
E	3ex-diff-domain	0.50	1.00	1.00	1.00	1.00	N/A†	0.90
Overall PPR		0.70	0.82	1.00	**0.38**	1.00	1.00‡	0.92
Pass rate		0/50	0/50	0/50	0/50	0/50	0/50	0/50

†`gpt-5.4-mini` var-E: 8 HTTP errors; $n_{\text{eff}} = 2$; excluded from per-variant PPR.
‡Overall PPR for `gpt-5.4-mini` computed over 4 valid variants (A–D); var-E excluded.

✓ Panel A: L3 — explicit rule (`gpt-4o`) [CORRECT]

System: "if-blocks run when condition is FALSE."

`gpt-4o` output **(CORRECT):**

```
if n > 1:              % inverted: runs when FALSE
    return n
return fib(n-1) + fib(n-2)
```

× Panel B: L4 — examples only (`gpt-4o`) [INCORRECT]

User: [2 worked examples with inverted semantics, no explicit rule]

`gpt-4o` output **(INCORRECT):**

```
if n <= 1:             % Python prior, not inverted
    return n
return fib(n-1) + fib(n-2)
```

Fig. 4: Prior dominance vs. pattern blindness isolated on the same model (`gpt-4o`). **A**: `gpt-4o` correctly inverts logic with explicit rule (L3). **B**: The same model reverts to Python prior when the rule is embedded only in examples (L4). This contrast isolates pattern blindness as a failure mode distinct from prior dominance.

E. L4 Multi-task Generalization

To assess whether pattern blindness extends beyond Fibonacci, we evaluated all seven models on three tasks: T1 (`fibonacci`), T2 (`is_sorted`), and T3 (`count_vowels`). The multi-task prompt uses inline code comments that partially annotate the inversion rule (*e.g.*, `# runs when a > b is FALSE, i.e., a <= b`); this is a *partially-annotated L4* condition—less explicit than L3 (no stated rule), but more informative than the pure example-only variants used in the ablation. `gpt-4o`, `gpt-4o-mini`, `gpt-5.4-mini`, and `o4-mini` were evaluated at $n = 10$ or $n = 15$ per task; `gpt-4.1-mini` at $n = 20$ per task; `gpt-4.1` and `gpt-4.1-nano` at $n = 10$ per task (255 initial runs). We further scaled `gpt-4o` T3 and `gpt-4.1` T2 to $n = 50$ each to resolve preliminary ambiguities, and extended `o4-mini` T2/T3 to $n = 20$ each, yielding 395 total multi-task runs. Results are in Table V.

Seven findings emerge.

(1) **T2 is extractable only by mid-to-large models**: `gpt-4o` (10/10) and `gpt-4.1-mini` (20/20) pass T2 fully, while `gpt-4.1` (1/50, PPR=0.42), `gpt-4.1-nano` (0/10), `gpt-5.4-mini` (0/10), and `gpt-4o-mini` (0/10) fail. This reveals a non-monotonic capacity threshold: T2 extractability does not scale linearly within or across model families.

(2) **T1 (Fibonacci) is resistant for all non-reasoning models**: six of seven models fail T1, with high or full PPR (PPR=1.0 for `gpt-4o`, `gpt-4.1`, `gpt-4.1-mini`, `gpt-5.4-mini`; PPR=0.2 for `gpt-4.1-nano`; template fragmentation for `gpt-4o-mini`). The exception is `o4-mini`, which achieves 8/15 under the partially-annotated prompt (after correcting a token budget truncation; see §)—consistent with reasoning-model capability to leverage partial annotation cues that dense models overlook.

(3) **Operational substitution dominates T3**: the scale-up (`gpt-4o` T3, $n = 50$) reveals operational substitution in 47/50 runs (94%; Wilson 95% CI: [83%, 99%])—the model applies arithmetic transformations (`count -= 1; return -count`) to achieve correct output without adopting the L4 semantic rule. The preliminary $n = 10$ result (1 pass) was a statistical artifact; at $n = 50$, `gpt-4o` achieves 0/50 pass (Wilson 95% CI: [0%, 7.1%]). `gpt-5.4-mini` T3 also exhibits operational substitution (PPR=0.0, all 10 runs), confirming this as a structurally distinct, frequent failure mode.

(4) **Model capacity determines T2 extractability**: `gpt-4o-mini`, `gpt-4.1`, `gpt-4.1-nano`, and `gpt-5.4-mini` all fail T2, while `gpt-4o` and `gpt-4.1-mini` succeed—consistent with the capacity threshold observed at L3.

(5) **Reasoning partially mitigates pattern blindness under partial annotation**: `o4-mini` achieves 8/15 on T1 under the partially-annotated prompt (after token budget correction), but fails T2 and T3 (0/20 each at extended $n = 20$; overall 8/55 across T1+T2+T3). Under the strict example-only ablation (§ V-D), `o4-mini` achieves 0/50—chain-of-thought reasoning alone cannot overcome pure pattern blindness, but reasoning models can exploit partial annotation cues that dense models miss.

(6) **Within-family inconsistency is confirmed**: `gpt-4.1` T2 scale-up ($n = 50$) confirms the capacity gap is systematic—1/50 pass (Wilson 95% CI: [0%, 10.5%]) vs. `gpt-4.1-mini` 20/20 (Fisher's exact $p < 0.0001$). Fine-tuning or RLHF choices appear to play a larger role than raw scale in determining L4 pattern extraction capability.

(7) **`gpt-5.4-mini` shows uniform prior dominance across all tasks**: 0/30 with T1 PPR=1.0, T2 PPR=1.0, and T3 operational substitution pattern, consistent with its L4 ablation PPR=1.0 on Fibonacci variants.

TABLE V: L4 multi-task generalization (partially-annotated prompts; not directly comparable to Table IV). Each cell shows pass rate; PPR in parentheses. "Pass" = model applied inverted if-block semantics correctly. T1=fibonacci, T2=is_sorted, T3=count_vowels. OpSub = operational substitution (correct output via arithmetic workaround, not semantic inversion).

	T1: fibonacci			T2: is_sorted			T3: count_vowels		
Model	n	**Pass**	**PPR**	n	**Pass**	**PPR**	n	**Pass**	**PPR**
`gpt-4o`	10	0/10	1.00	10	**10/10**	0.00	50[‡]	0/50	0.00[‡]
`gpt-4o-mini`	10	0/10[*]	0.00	10	0/10	1.00	10	0/10	0.00
`gpt-4.1`	10	0/10	1.00	50[†]	1/50	0.42	10	0/10	0.00
`gpt-4.1-mini`	20	0/20	1.00	20	**20/20**	0.00	20	0/20	0.25
`gpt-4.1-nano`	10	0/10	0.20	10	0/10	1.00	10	0/10	0.00
`gpt-5.4-mini`	10	0/10	1.00	10	0/10	1.00	10	0/10	0.00
`o4-mini`	15	**8/15**[§]	0.33	20[¶]	0/20	0.30	20[¶]	0/20	0.00

[*]Template fragmentation (variant of prior dominance). [†]Wilson 95% CI: pass [0%, 10.5%], PPR [29%, 56%]; Fisher's exact $p < 0.0001$ vs. `gpt-4.1-mini`. [‡]OpSub in 47/50 runs (94%, CI: [83%, 99%]). [§]After token budget correction (`max_completion_tokens` 400→5000); see §V-D. [¶]Scale-up: `o4-mini` T2/T3 extended from $n = 15$ to $n = 20$; T2 PPR=0.30, T3 PPR=0.00.

F. Three Failure Modes

- **Prior Dominance** (L1, L3-weak, L4-T2-mini, L4-T1-all): stated or implicit rule is overridden by pretrained token distribution.
- **Pattern Blindness** (L4-T1): model cannot extract the semantic rule from examples alone, even when the same model follows the rule when stated explicitly (L3). T1 (Fibonacci) shows universal pattern blindness across all models.
- **Operational Substitution** (L4-T3-`gpt-4o`, $n = 50$): model compensates via arithmetic output transformation rather than adopting the semantic inversion rule, indicating goal awareness without rule internalization. Confirmed as dominant at 94% of T3 `gpt-4o` runs (Wilson 95% CI: [83%, 99%]).

We also observe *template fragmentation* in `gpt-4o-mini` T1: the model partially disrupts its standard template but does not achieve semantic inversion, yielding an incorrect but structurally novel output. We treat this as a sub-variant of prior dominance rather than a distinct fourth mode, given the small sample ($n = 10$); further study at larger n may warrant reclassification.

Task difficulty interacts with model capacity to determine which failure mode dominates. T2 (is_sorted) is extractable by mid-to-large models; T1 (Fibonacci) is universally resistant; T3 (count_vowels) triggers operational substitution as the dominant escape strategy for capable models.

Failure taxonomy across 419 failed L4 runs (ablation + multitask T1): Type-I (exact Python prior) accounts for 83.1% of failures, Type-II (partial attempt) for 12.6%, and Type-III (operational substitution) for 4.3%. Type-I dominates in `gpt-4.1` and `gpt-4.1-nano` (100% of failures); Type-II concentrates in `gpt-4.1-mini` (44%) and `gpt-4o-mini` (32%); Type-III appears primarily in `gpt-4o` (25% of its failures).

G. CoT Ablation: Reasoning About Inversion Is Not Sufficient

We tested whether chain-of-thought (CoT) prompting overcomes pattern blindness by adding "think step by step about the pattern first" to the L4 Variant A prompt ($n = 20$ per condition, two models).

Result: CoT does not improve pass rate. Both `gpt-4o` and `gpt-4.1-mini` achieve 0/20 in both CoT and no-CoT conditions. However, CoT dramatically changes the *reasoning behavior*:

- `gpt-4o`: CoT reduces PPR from 1.00 to 0.15 and raises *mentions_inversion* to 100%—the model explicitly *describes* the inverted pattern in its reasoning chain but still generates standard Python code.
- `gpt-4.1-mini`: CoT reduces PPR from 0.40 to 0.00 with 100% mentions_inversion rate—same dissociation between reasoning and code generation.

This reveals a *reasoning–generation dissociation*: models can articulate the semantic inversion rule when prompted to reason, yet their code generation module defaults to the Python prior regardless. Pattern blindness is thus not a failure to *detect* the pattern but a failure to *override* the prior during generation.

H. Open-Weight Models: Preliminary L1 and L4 Results

To assess whether the failure modes generalize beyond the OpenAI model family, we conducted preliminary evaluations on two open-weight models via Groq API: `Llama-3.3-70B-Versatile` and `Qwen3-32B`.

L1 (ctx-pack, $n = 20$): `Llama-3.3-70B` achieves KLR=0.23 (PSS=0.34), nearly matching the OpenAI best (`gpt-4.1-mini` ctx-pack, 0.21). `Qwen3-32B` achieves KLR=1.00 (PSS=0.00): all 20 outputs contained at least one native Python keyword with no valid code skeleton—the worst L1 result across all tested models. The inter-model gap (0.23 vs. 1.00) exceeds the intra-family spread of `gpt-4.x` models (0.21–0.69), indicating that instruction-following fine-tuning rather than parameter count governs alias compliance.

L4 Variant A ($n = 20$): Both open-weight models fail. `Llama-3.3-70B`: 0/20 pass, PPR=0.05 (1/20 strict Python prior; remaining 19 outputs applied standard control flow without inversion). `Qwen3-32B`: 0/18 pass ($n_{\text{eff}} = 18$; 2 HTTP errors), PPR=1.00 (complete prior dominance). Neither model triggered inversion detection (noticed_inversion=0).

These results extend the pattern blindness finding beyond OpenAI models.

Limitations: Pilot results ($n = 20$, Variant A only, cloud inference via Groq with rate-limit retries); full multi-variant ($n = 50$) and L2/L3 evaluations are deferred.

VI. Discussion

A. Effective LLM-resistant language design

Our results identify a principled recipe for LLM-resistant programming languages:

1) Token-level perturbation alone is insufficient (models partially resist, but at measured rates).
2) Syntactic restructuring is also insufficient (models adopt explicit structural rules).
3) Semantic inversion with explicit instruction provides capacity-dependent resistance.
4) Semantic inversion without explanation (L4) provides consistent resistance via pattern blindness.
5) **Task selection is critical**: tasks with deep recursive priors (e.g., Fibonacci) are universally resistant, while tasks with shallower or more varied idioms (e.g., is_sorted) remain extractable by mid-to-large models. Instructors should prefer tasks with strong, uniform Python prior entrenchment.

The most practical target for educational use is L4: retain Python surface syntax, invert conditional semantics, and provide only worked examples rather than explicit rules—applied to tasks with deep Python priors.

B. Implications for programming education

Our findings partially address the concern of LLM-solved homework. L4-style assignments provide meaningful resistance for tasks with deep Python priors (e.g., Fibonacci recursion): all non-reasoning models fail T1 (0/10 to 0/20 pass rates), though via different failure modes: prior dominance (PPR=1.0) for most models, partial template fragmentation (PPR=0.20) for `gpt-4.1-nano`; o4-mini achieves 8/15 only under the partially-annotated multi-task prompt (not under the strict example-only condition). However, task selection matters: T2 (is_sorted) is solved by capable models (`gpt-4o`, `gpt-4.1-mini` at 100%), indicating that simpler idioms with shallow priors may remain vulnerable. The general mechanism for LLM failure on L4 is: (1) the model defaults to Python prior when encountering familiar syntax, and (2) even when examples are provided, pattern extraction of semantic inversion is beyond current few-shot capability for sufficiently prior-entrenched patterns.

Students who understand the L4 rules (from lecture instruction) *should* retain an advantage over LLM-assisted submissions in principle, since current models universally fail L4. However, this claim requires empirical validation: a formal human study (Exp-5, planned) is needed to confirm that L4 tasks do not also confuse human learners, and that the human–LLM performance gap is large enough to restore discriminative power in practice.

C. Limits

While the multi-task experiment (Section V) covers three tasks and seven models (355 total runs including scale-ups), the task set is limited. The task-dependent pass rate for `gpt-4o` on T2 (is_sorted: 10/10 pass vs. T1 fib: 0/10) confirms that L4 resistance is not unconditional—tasks with shallow or varied Python idioms remain extractable by capable models. The PPR metric for L3/L4 is task-specific and requires per-task oracle semantics. Human baseline experiments (Exp-5) are needed to confirm that L4 does not also confuse human learners.

The *operational substitution* failure mode is confirmed as dominant for T3 at $n = 50$: 47/50 `gpt-4o` runs (94%; Wilson 95% CI: [83%, 99%]). Our structural judge detects known arithmetic patterns (e.g., `count -= 1; return -count`); novel workarounds not matching these heuristics may evade detection. An execution-based judge would provide exhaustive coverage of this failure mode. The scale-up results establish it as real and frequent, not a one-off artifact.

The L4 ablation ($n = 50$ per model, 350 total runs) strengthens the pattern blindness finding: zero pass rate holds across all five prompt variants and all seven models including the reasoning model `o4-mini`. One important caveat: `o4-mini` was evaluated at temperature=1 (an API constraint), while all other models used temperature=0 (deterministic). Cross-model comparisons involving `o4-mini` should therefore be interpreted with caution, as sampling variance at temperature=1 may contribute to PPR variance across variants.

The high PPR variance across variants (e.g., `gpt-4.1-mini` variant B PPR=0.0 vs. variant E PPR=1.0) warrants caution—some configurations may trigger different failure sub-modes that are not yet fully characterized. A parallel run of `gpt-5.4-mini` completed $n = 42$ valid evaluations (8 HTTP errors in variant E); all valid runs show PPR=1.0 and pass=0, consistent with full prior dominance across all variants. The overall PPR of 1.00 for `gpt-5.4-mini` represents the strongest prior dominance among all tested models.

Notably, an earlier preliminary run of `o4-mini` at $n = 20$ yielded 1/20 pass; at $n = 50$, this result does not replicate (0/50), indicating that the earlier observation was likely a statistical artifact and not evidence of systematic reasoning-based inversion.

Corrected `o4-mini` result: partial annotation enables reasoning-model extraction. An earlier multi-task run of `o4-mini` used `max_completion_tokens=400`, which truncated reasoning-model responses before the code block was emitted. After correcting to `max_completion_tokens=5000`, `o4-mini` achieves 8/15 on T1 (partially-annotated prompt). This revision has two implications: (a) the 0/45 result from the strict example-only ablation (Table IV) is unaffected and confirms pattern blindness under pure implicit delivery; (b) under partial annotation (inline comments that partially articulate the rule), `o4-mini`'s reasoning capability provides a meaningful advantage that dense models cannot replicate. We report both results transparently. This suggests that the boundary between

L3 (explicit rule) and L4 (implicit) is not binary—degrees of partial annotation interact with model reasoning depth.

D. Next steps

- Exp-4 (partial): Multi-task results acquired for T2/T3; broader task coverage (nested control-flow, stateful logic) remains for future work.
- Exp-5: Human study comparing student performance vs. LLM performance on L4 tasks; critical for validating the educational claim.
- Operational substitution: an execution-based judge would improve detection precision beyond the current structural heuristic, particularly for novel arithmetic workarounds beyond the `count -= 1` pattern.
- L4 difficulty scaling: graded version with more/fewer examples, measuring the tipping point at which models begin to extract the inversion pattern.

VII. Conclusion

We presented a four-level taxonomy (L1–L4) for designing LLM-resistant programming languages, grounded in the NTP prior structure of language models.

Our central finding is that resistance does not come from surface-level token changes or structural novelty—both of which LLMs can adapt to with explicit instruction. Robust resistance requires semantic inversion: the same Python-like surface, but inverted conditional semantics. When semantic inversion is delivered implicitly (via examples only), all tested models exhibit pattern blindness and default to Python priors.

We identify three distinct failure mechanisms: *prior dominance*, where the prior overrides the stated rule; *pattern blindness*, where the model fails to extract the semantic rule from examples; and *operational substitution*, where the model achieves correct output via arithmetic workaround rather than adopting the semantic inversion rule. These mechanisms have different implications for language design: prior dominance can be partially mitigated by explicit context packs; pattern blindness requires implicit delivery; operational substitution may require execution-based evaluation to detect.

These findings provide a principled basis for designing programming course assessments that remain meaningful in the LLM era, where LLMs will fail on L4-style problems even as they trivially solve standard Python assignments.

VIII. Prompt and Evaluation Protocol

A. L1: Token Alias Substitution

Alias sampling strategy. Representative mappings from three prompt versions:

Token	v1 alias	v35 alias	v90 alias
`if`	`return`	anti-prior phrase	long natural phrase
`def`	`if`	`def`	`return`
`for`	long phrase	`xk`	long phrase
`return`	long phrase	long phrase	`out`
`print`	`do`	—	`qn`

S0 — Baseline system prompt (L1)

"Solve the given prompt exactly. Return ONLY code (no explanation, no markdown). Follow alias mapping strictly."

S1 — Context-pack system prompt (L1, ablation)

"Solve the given prompt exactly. Return ONLY code. Follow alias mapping strictly."

[Transformed language contract]
Use transformed aliases as listed in the prompt. Do NOT output original Python keywords for transformed entries. Output only executable code in the transformed language.

Fig. 5: L1 system prompts. S0 is the compact baseline; S1 adds an explicit alias contract to test whether contextual reinforcement reduces prior dominance.

L1 user prompt template

[Language Description]
Python grammar remains unchanged.
Keyword aliases are user-configurable strings.
At runtime, detected aliases are normalized back to Python.

Current mapping summary:
`if` → {alias} `in` → {alias} `def` → {alias}
`for` → {alias} `return` → {alias} `print` → {alias}

[Task]
1) Define function `fib(n)`.
2) If `n <= 1`, return `n`.
3) Otherwise return `fib(n-1) + fib(n-2)`.
4) Loop `i = 0..6`, print `(i, fib(i))`.

Fig. 6: L1 user prompt template. Alias values are varied across v1–v120, using tokens of differing length, semantic distance, and collision potential. The task is fixed as Fibonacci to control task difficulty.

L2 grammar specification (complete)

This is NOT Python. All Python keywords are FORBIDDEN.

`:define name [args] ->` (replaces `def name(args):`)
`<= value` (replaces `return value`)
`value ?cond: res ;else: alt ;end` (replaces `if`)
`loop var over iterable { body }` (replaces `for`)
`;;` closes function block

Fig. 7: L2 grammar specification as given to the model. Models must produce code using exclusively these constructs. Result: all models fully adopted the syntax (SIR=0.0), showing LLMs readily follow explicit structural rules.

B. L2: Syntax Inversion

C. L3: Semantic Inversion (Explicit)

D. L4: Semantic Inversion (Implicit)

E. Evaluation Metrics

- **KLR** (Keyword Leakage Rate): fraction of alias slots where original Python keyword was used. ACR $= 1 - \text{KLR}$

L3 system prompt — explicit semantic rule

*"Follow the L3 language rules exactly. If-blocks run when condition is **FALSE**."*

L3 user prompt (complete)

L3 rule: Syntax is identical to Python, but:
RULE: Indented block executes when condition is **FALSE**.
RULE: Else-block executes when condition is **TRUE**.

Example:

```
if x > 5:
    print("small")  # runs when NOT x > 5
else:
    print("large")  # runs when x > 5
```

Task: Write `fib(n)` in L3. Intended logic: if $n \leq 1$ return n; else return `fib(n-1)+fib(n-2)`.
Think carefully: your if-block runs when condition is FALSE.

Fig. 8: L3 prompts. The rule is stated explicitly in the system message. Weaker models (gpt-4o-mini) still fail (PPR=1.0), while stronger models succeed—revealing a capability threshold for semantic meta-rule following.

L4 user prompt (example-only delivery, no explicit rule)

Here are some examples in our programming language:

Example 1 — max function:

```
def max_val(a, b):
    if a > b:
        return b   % smaller when a > b
    return a
```

Output: `max_val(3,5)=3`, `max_val(7,2)=2`

Example 2 — even check:

```
def is_even(n):
    if n % 2 == 0:
        return False
    return True
```

Output: `is_even(4)=False`, `is_even(3)=True`

Task: Write `fib(n)` in this language. Target: `fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`.
Print `fib(0)` through `fib(6)`.

Fig. 9: L4 prompt (no system message). The semantic inversion rule is not stated; models must infer it from worked examples. Even gpt-4o, which succeeds at L3, defaults to canonical Python fib—exhibiting *pattern blindness*.

is a derived quantity.

- **PSS** (Partial Structural Score): weighted sub-component credit (alias: 0.4, parse: 0.25, skeleton: 0.25, format: 0.1).
- **PPR** (Python Prior Rate, L3/L4): fraction of outputs using Python-style condition without inversion.

F. Judge Protocol (L1)

1) Alias compliance: check that alias is present and original token is absent.
2) Normalization: replace aliases → Python keywords via word-boundary regex.
3) Parse check: `ast.parse()` on normalized code.
4) Skeleton check: verify `fib`, base case, recursion, and print loop.

G. Judge Protocol (L3/L4)

- **Python-prior**: model writes `if n <= 1: return n` (non-inverted) ⇒ PPR=1 (fail).
- **L3-correct**: model writes `if n > 1: return n` or equivalent ⇒ PPR=0 (pass).

IX. Complete Experiment Log

A. All experiments (chronological)

1) **E1–E4: L1 baseline, $n = 120$, 4 models (2026-03-23).** Models: gpt-4o-mini, gpt-4.1-mini, gpt-4.1-nano, gpt-5.4-mini. Files: `batch-v1-120.`*`model`*`.json`
2) **E5: L1 context-pack ablation, $n = 120$, gpt-4o-mini (2026-03-23).** Setting: CONTEXT_PACK=1. KLR: 0.423 → 0.260 (39% reduction from E1).
3) **E6: L1 baseline, $n = 120$, gpt-4o (2026-03-24).** KLR=0.366, highest structural compliance.
4) **Partial judge applied (2026-03-24).** Script: `partial_judge.py` applied to all L1 batches.
5) **E7: L2 syntax inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit grammar rules provided in system prompt. SIR=0.0; model adopted L2 syntax fully.
6) **E8: L3 semantic inversion, $n = 5$, gpt-4o-mini (2026-03-24).** Explicit inversion rule in system prompt. PPR=1.0: prior dominance, complete failure.
7) **E9–E10: L3 semantic inversion, $n = 10$, gpt-4o and gpt-4.1-mini (2026-03-24).** Both models: PPR=0.0 (successfully followed rule).
8) **E11: L4 implicit semantic, $n = 5$, gpt-4o (2026-03-24).** No system prompt; only 2 worked examples shown. Pattern blindness observed: model ignored inversion cue.
9) **E12: L1 context-pack, $n = 20$, gpt-4.1 family + o4-mini (2026-03-25).** Models: gpt-4.1 (KLR=0.295), gpt-4.1-mini (KLR=0.210), gpt-4.1-nano (KLR=0.378), o4-mini (KLR=0.265). Files: `l1-gpt41-contextpack-2026-03-25.json`, etc. Note: o4-mini requires `temperature=1` (API constraint).
10) **E13: L4 ablation, $n = 50$, 5 models (2026-03-25).** Models: gpt-4o, gpt-4o-mini, gpt-4.1-mini, gpt-5.4-mini; later gpt-4.1 and gpt-4.1-nano scaled to $n = 50$. Result: 0/50 pass for all five variants × all models. File: `L4-ablation-n50.`*`model`*`.2026-03-25.json`
11) **E14: L4 ablation, $n = 50$, o4-mini (2026-03-25).** Reasoning model; confirms 0/50 pass at scale (1/20 preliminary was a statistical artifact). File: `L4-ablation-n50.o4mini.2026-03-25.json`

12) **E15: L4 multi-task, $n = 10$–20 per task, 7 models (2026-03-25).** Tasks: T1 (fibonacci), T2 (is_sorted), T3 (count_vowels). Models: gpt-4o, gpt-4o-mini, gpt-4.1, gpt-4.1-mini, gpt-4.1-nano, gpt-5.4-mini, o4-mini (255 total runs). gpt-5.4-mini: 0/30 (T1 PPR=1.0, T2 PPR=1.0, T3 operational substitution). Files: `L4-multitask.model.2026-03-25.json`
13) **E16: L4 multi-task scale-up, $n = 50$, gpt-4o T3 and gpt-4.1 T2 (2026-03-25).** Rationale: preliminary $n = 10$ results showed 1 pass each; scaled to $n = 50$ to rule out statistical artifact. gpt-4o T3 (`count_vowels`): 0/50 pass; operational substitution in 47/50 runs (94%, Wilson 95% CI: [83%, 99%]). Confirms preliminary pass was artifact. gpt-4.1 T2 (`is_sorted`): 1/50 pass (Wilson 95% CI: [0.0%, 10.5%]); Fisher's exact vs. gpt-4.1-mini 20/20: $p < 0.0001$. Total multi-task runs (E15+E16): 355 (255 initial + 100 scale-up). Files: `L4-scaleup-count_vowels.gpt4o.n50.2026-03-25.json`, `L4-scaleup-is_sorted.gpt41.n50.2026-03-25.json`. Script: `scripts/run_targeted_scaleup.py`

B. API and runtime settings

Setting	Value
Temperature	0
Base URL	`https://api.openai.com/v1`
Timeout	120s
Prompt delay	0.4s
API key source	Project environment variable
Retry	None

C. Repository structure for replication

Script / Path	Purpose
`run_gpt54_eval.py`	L1 batch runner
`run_l2_eval.py`	L2 batch runner
`run_l4_ablation.py`	L4 ablation runner
`run_l4_multitask.py`	L4 multi-task runner
`run_targeted_scaleup.py`	L4 targeted scale-up ($n = 50$)
`partial_judge.py`	Continuous metrics
`compare_eval_runs.py`	Run comparison
`docs/archive/*/`	Prompt banks (L1–L4)
`docs/research/results/`	All output JSONs
`language-levels/`	Level specs

X. Appendix: Prompt Samples, Taxonomy, and Config Snapshot

A. Prompt samples (readable form)

Representative prompt-map blocks from the versioned bank:

Sample A (v1)

- if → return
- def → if
- for → long natural phrase (renamed symbol)
- in → long natural phrase (strict compilation token)
- return → long natural phrase (keyword mapping token)
- print → do

Sample B (v35)

- in → mapped token phrase
- def → def
- for → xk
- return → long phrase preserving grammar
- if → anti-prior phrase

Sample C (v90)

- def → return
- for → phrase mapped back to reserved token
- in → exact mapping phrase
- return → out
- if → long natural phrase
- print → qn

B. Violation taxonomy summary (gpt-4o-mini, v1–v120)

From the context-pack comparison artifact, dominant violation categories are:

- Original keyword leakage (e.g., `for`, `if`, `return`, `in`, `def`)
- Parse failure after normalization (line-specific syntax failure)
- Missing alias emissions for required transformed tokens

Top counts in context-pack run include:

- `for` leaked: 52
- parse failed (line 1): 35
- `if` leaked: 33

C. Execution config snapshot (non-secret)

TABLE VI: Key runtime settings

Setting	Value
Models	gpt-4o, gpt-4o-mini, gpt-4.1, gpt-4.1-mini, gpt-4.1-nano, gpt-5.4-mini, o4-mini
L1 range	v1–v120 (baseline); v1–v20 (ctx-pack for 4.1/o4-mini)
CONTEXT_PACK	0/1
Temperature	0 (all models except o4-mini); 1 (o4-mini, API requirement)
Judge	deterministic

Secret-bearing values (API keys) are excluded and loaded via runtime environment. Language specs are in `language-levels/` (repository).

XI. Reproducibility and Artifact Availability

All experiments are committed to the repository. Representative paths:

- Results: `docs/research/results/`
- Language specs: `language-levels/`
- Evaluation scripts: `scripts/partial_judge.py`, `scripts/run_l2_eval.py`
- Paper: `paper/overleaf/`

REFERENCES

[1] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[3] T. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," arXiv:2406.15079, 2024.

[4] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," arXiv:2403.07974, 2024.

[5] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *EMNLP-Findings*, 2020.

[6] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models," in *EMNLP*, 2021.

[7] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," 2022.

[8] Z. Luo *et al.*, "WizardCoder: Empowering code large language models with evol-instruct," arXiv:2306.08568, 2024.

[9] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," arXiv:2401.14196, 2024.

[10] Y. Liu *et al.*, "CodeIF: Benchmarking instruction-following in code generation," arXiv:2502.19166, 2025.

[11] W. Zhang *et al.*, "MultiCodeIF: Comprehensive evaluation of code instruction-following," arXiv:2507.00699, 2025.

[12] E. Chen *et al.*, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024.

[13] J. Xie *et al.*, "Adaptive chameleon or stubborn sloth: Revealing the behavior of LLMs in knowledge conflicts," ICLR 2024 (arXiv:2305.13300), 2024.

[14] S. Wu *et al.*, "Untangle the KNOT: Interweaving conflicting knowledge and reasoning skills in LLMs," arXiv:2403.08319, 2024.

[15] Z. Zhang *et al.*, "Knowledge conflicts for LLMs: A survey," EMNLP 2024 (arXiv:2403.08319), 2024.

[16] F. Shi *et al.*, "Large language models can be easily distracted by irrelevant context," ICML 2023 (arXiv:2302.00093), 2023.

[17] Z. Wang *et al.*, "A law of next-token prediction in large language models," arXiv:2408.13442, 2024.

[18] S. Min *et al.*, "Rethinking the role of demonstrations: What makes in-context learning work?" EMNLP 2022 (arXiv:2202.12837), 2022.

[19] Q. Dong *et al.*, "A survey on in-context learning," arXiv:2301.00234, 2023.

[20] A. Lampinen *et al.*, "A-Not-B errors in LLMs: Limitations of in-context learning," arXiv:2409.15454, 2024.

[21] Q. Zhao *et al.*, "LLMs struggle to perform counterfactual reasoning with parametric knowledge," arXiv:2410.06392, 2024.

[22] M. Turpin *et al.*, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," NeurIPS 2023 (arXiv:2305.04388), 2023.

[23] K. Valmeekam *et al.*, "LLMs still can't plan; can LRMs? evaluation of OpenAI's o1," arXiv:2409.13373, 2024.

[24] K. Zhu *et al.*, "PromptBench: Evaluating robustness of large language models," arXiv:2306.04528, 2023.

[25] R. Pan *et al.*, "Attacking LLMs with code generation: Semantically-aligned adversarial perturbations," arXiv:2412.16359, 2024.

[26] S. Jha *et al.*, "Break the chain: Reasoning failures in LLMs via adversarial code prompting," arXiv:2506.06971, 2024.

[27] A. Khan *et al.*, "Defeating LLMs with adversarial perturbations in programming assignments," arXiv:2410.09318, 2024.

[28] J. Wu *et al.*, "A survey on LLM-based code generation for low-resource and domain-specific languages," arXiv:2410.03981, 2024.

[29] W. Chen *et al.*, "ConCodeEval: Evaluating LLMs for code constraint comprehension in DSLs," EMNLP 2024, 2024.

[30] A. Ni *et al.*, "L2CEval: Evaluating language-to-code generation capabilities," TACL 2024, 2024.

[31] Y. Dong *et al.*, "XGrammar: Flexible and efficient structured generation for LLMs," arXiv:2411.15100, 2024.

[32] J. Muendler *et al.*, "Type-constrained code generation with language models," PLDI 2025, 2025.

[33] C. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" 2024.

[34] A. Dakhel *et al.*, "GitHub Copilot AI pair programmer: Asset or liability?" 2023.

[35] S. Lau and P. Guo, "From coding assistant to software developer: How novice programmers use LLMs," arXiv:2304.01984, 2023.

[36] J. Prather *et al.*, "It's complicated: Perspectives of novice programmers on GitHub Copilot," arXiv:2303.08733, 2023.

[37] M. Liffiton *et al.*, "CodeHelp: Using large language models with guardrails for programming classes," arXiv:2308.06921, 2023.

[38] E. Novak and B. McDanel, "Designing LLM-resistant programming assignments," SIGCSE TS 2025, 2025.

[39] M. Orenstrakh *et al.*, "Detecting LLM-generated code assignments using perplexity of code," arXiv:2307.09714, 2023.

[40] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," NeurIPS 2022 (arXiv:2203.02155), 2022.